

页 注 榜 栏

iOS使用VOIP与CallKit实现体验优质的网络通讯功能 原创

琿少 2021-10-21 17:51:28 ©著作权

文章标签 CallKit VoIP iOS网络电话 iOS识别联系人 iOS电话黑名单 文章分类 代码人生 阅读量 276

iOS使用VOIP与CallKit实现体验优质的网络通讯功能

VOIP是Apple提供给开发者的网络电话功能接口。简单来说，其可以让你的应用程序在完全杀死的情况下被服务端唤醒。CallKit是iOS10引入的新框架，使用它可以让你的应用程序调用系统的通话和通话记录界面。试想一下，用户可以在锁屏，应用被杀死，应用在后台等情况下收到通讯请求并且弹出系统的通话界面进行交互是多么酷的一件事。

一、创建VOIP推送证书

VOIP说是一种网络电话服务，其实质是一种特殊的长连接，使用它每个网络电话类APP不需要自己单独进行保活维护，在进行通话请求时，只需要发送一条VOIP推送，VOIP推送会将应用程序拉起，之后由应用程序处理通讯逻辑。VOIP也是Push的一种，只是其是一种特殊的Push，普通的Push当应用被杀死后可以收到，但是用户点击Push消息前应用程序是不会被激活的，VOIP则不然，可以直接激活应用。

VOIP推送证书的创建方式与普通推送证书的创建方式基本一致，首先需要生成certSigningRequest文件，打开钥匙串应用：



在证书助理栏选择从证书颁发机构申请证书：



填写相关资料后，将生成的文件保存：



琿少

485 原创

503.4万 人气

32 粉丝

0 翻译

0 转载

1 关注



+ 关注

私信

近期文章

- 1.PostgreSQL 主从复制读写分离
- 2.c-fmt-fn标签用法
- 3.分布式金融架构?构课(1)-如何成
- 4.电子工程师如何经营人脉_FPC
- 5.网站建设必须明白的宗旨，助

文章目录

iOS使用VOIP与CallKit实现体

- 一、创建VOIP推送证书
- 二、PushKit详析
- 三、关于CallKit框架

1.先来看CXProvider类

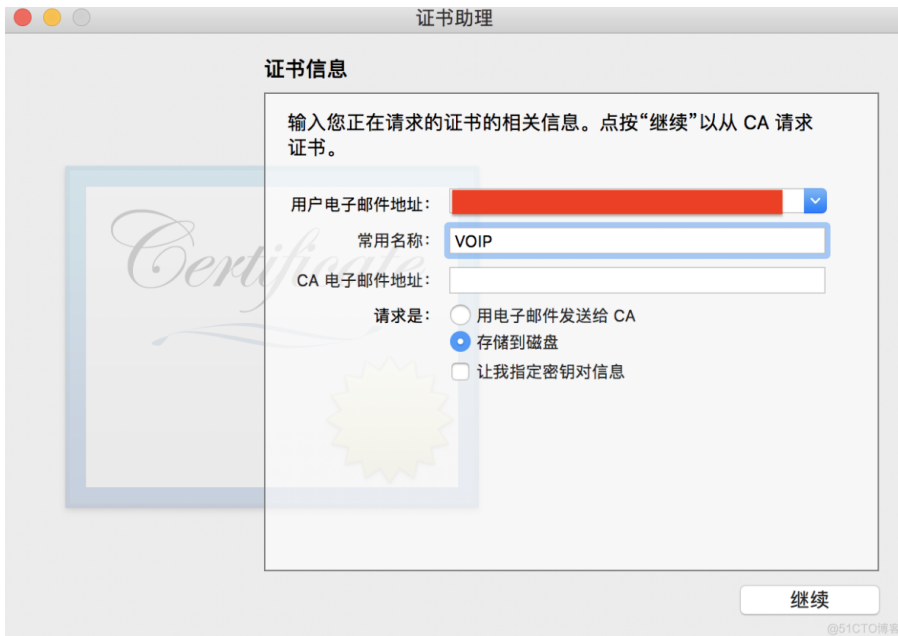
2.在看CXProviderConfigur

3.CXCallUpdate类

4.CXProviderDelegate相关

5.CXCallController解析

6.CXTransaction类
- 四、进行来电拦截与号码识别



在Apple开发者中心创建新的证书，证书类型选择生产环境的VOIP服务证书：

Production

- ☐ **App Store and Ad Hoc**
Sign your iOS app for submission to the App Store or for Ad Hoc distribution.
- ☐ **Apple Push Notification service SSL (Sandbox & Production)**
Establish connectivity between your notification server, the Apple Push Notification service sandbox, and production environments to deliver remote notifications to your app. When utilizing HTTP/2, the same certificate can be used to deliver app notifications, update ClockKit complication data, and alert background VoIP apps of incoming activity. A separate certificate is required for each app you distribute.
- ☐ **Pass Type ID Certificate**
Sign and send updates to passes in Wallet.
- ☐ **Website Push ID Certificate**
Sign and send updates for Websites.
- ☐ **WatchKit Services Certificate**
Establish connectivity between your notification server, the Apple Push Notification service sandbox, and production environment to update ClockKit complication data. When utilizing HTTP/2, the same certificate can be used to deliver app notifications, update ClockKit complication data, and alert background VoIP apps of incoming activity. A separate certificate is required for each app you distribute.
- ☐ **VoIP Services Certificate**
Establish connectivity between your notification server, the Apple Push Notification service sandbox, and production environment to alert background VoIP apps of incoming activity. A separate certificate is required for each app you distribute.
- ☐ **Apple Pay Payment Processing Certificate**
Decrypt app transaction data sent by Apple to a merchant/developer.

@51CTO博客

需要注意，普通的推送分开发环境和生产环境，VOIP证书不进行区分，生产环境和开发环境是通用的。之后选择一个AppID并且上传前面生成的certSigningRequest文件来完成VOIP证书的创建。

创建完成后，在证书列表可以看到多了一个VOIP服务证书，可以加载此证书进行VOIP推送。



二、PushKit详析

我们知道，客户端若想要接收普通的Push消息，是需要注册Token，通过Token来进行个推的。VOIP推送也是一样的，只是这类推送需要使用PushKit框架。

首先需要用到PKPushRegistry类，这个类进行推送的相关配置和Token的申请：

```
1. @interface PKPushRegistry : NSObject
```

```

2.    //代理对象
3.    @property (readwrite,weak,nullable) id<PKPushRegistryDelegate> delegate;
4.    //目标推送类型
5.    /*
6.    PK_EXPORT PKPushType const PKPushTypeVoIP NS_AVAILABLE_IOS(8_0); //VOIP推送
7.    PK_EXPORT PKPushType const PKPushTypeComplication NS_AVAILABLE_IOS(9_0); //Watch更新
8.    PK_EXPORT PKPushType const PKPushTypeFileProvider NS_AVAILABLE_IOS(11_0); //文件传输
9.    */
10.   @property (readwrite,copy,nullable) NSSet<PKPushType> *desiredPushTypes;
11.   //获取本地缓存的Token 申请Token执行回调后 这个方法可以直接获取缓存
12.   - (nullable NSData *)pushTokenForType:(PKPushType)type;
13.   //初始化, 并设置工作线程
14.   - (instancetype)initWithQueue:(nullable dispatch_queue_t)queue NS_DESIGNATED_INITIALIZER;
15.   - (instancetype)init NS_UNAVAILABLE;
16.   @end

```

PKPushRegistryDelegate相关函数意义如下：

```

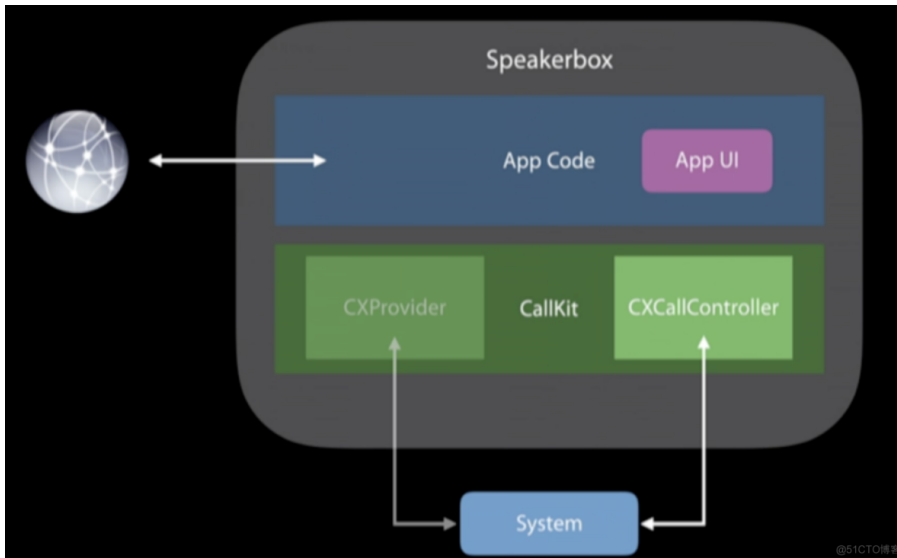
1.    //申请Token更新后回调
2.    /*
3.    PKPushCredentials是证书对象, 其中属性如下:
4.    @property (readonly,copy) PKPushType type; //推送类型
5.    @property (readonly,copy) NSData *token; //Token
6.    */
7.    - (void)pushRegistry:(PKPushRegistry *)registry didUpdatePushCredentials:(PKPushCre
8.    //收到推送后执行的回调
9.    /*
10.   PKPushPayload为推送信息 其中属性如下:
11.   @property (readonly,copy) PKPushType type; //推送类型
12.   @property (readonly,copy) NSDictionary *dictionaryPayload; //服务端发来的信息
13.   */
14.   - (void)pushRegistry:(PKPushRegistry *)registry didReceiveIncomingPushWithPayload:(C
15.   //作用同上, 最后的block需要在逻辑处理完成后主动回调
16.   - (void)pushRegistry:(PKPushRegistry *)registry didReceiveIncomingPushWithPayload:(C
17.   //Token失效的回调
18.   - (void)pushRegistry:(PKPushRegistry *)registry didInvalidatePushTokenForType:(PKPu

```

如果配置成功, 在收到VOIP推送时, 无论应用程序是否活跃, 都会执行代理函数, 我们便可以在其中进行逻辑处理。

三、关于CallKit框架

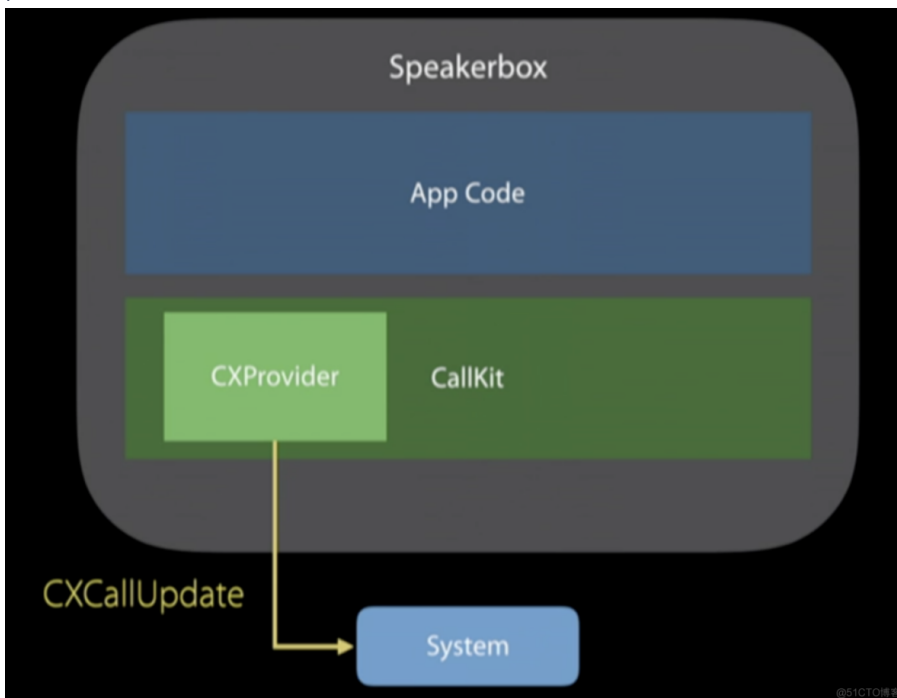
CallKit框架是iOS10后系统提供的一套网络电话UI和交互相关接口, 应用程序可以调用系统的电话界面来进行逻辑传递。下图比较形象的表达了应用程序与CallKit的关系:



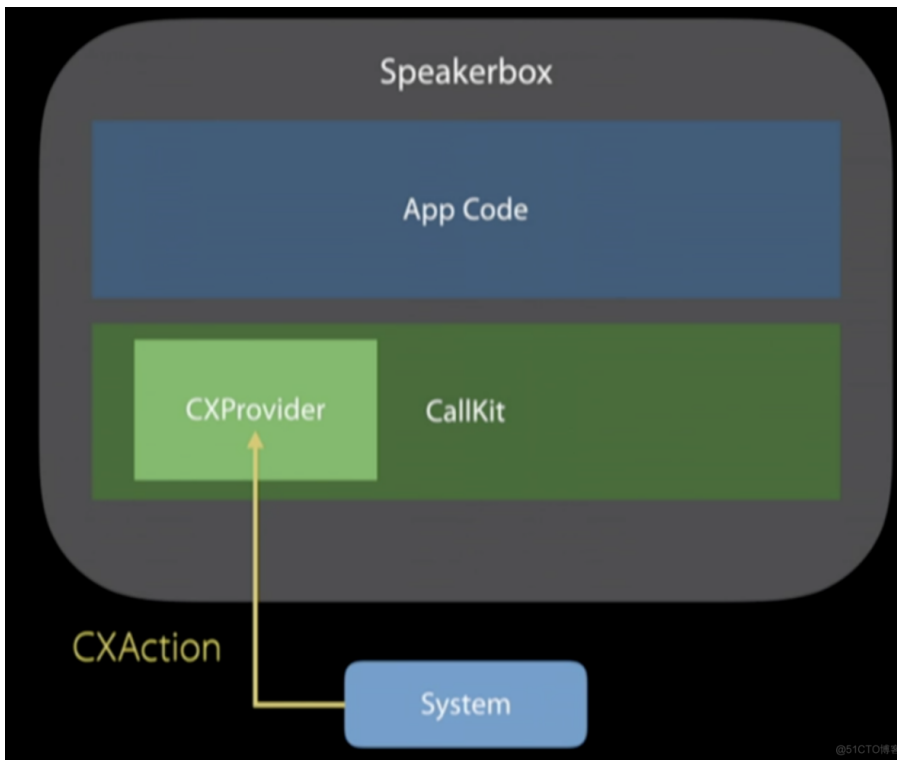
以收到网络电话为例，如果应用程序在前台，客户端可以直接处理通讯逻辑，如果应用程序不在前台，服务端可以发送一条VOIP推送唤醒APP，之后APP通知CallKit框架来唤起系统的通讯界面。CXProvider类主要负责系统服务于APP之间的交互。例如可以通过它来更新通话界面，显示通话的来自方，当用户点击通话界面的某些按钮后，也通过它来通知APP做逻辑处理。

需要注意，上图在CallKit和System之间有两个双向的白色箭头，这描述了CallKit和系统交互的四个方向。

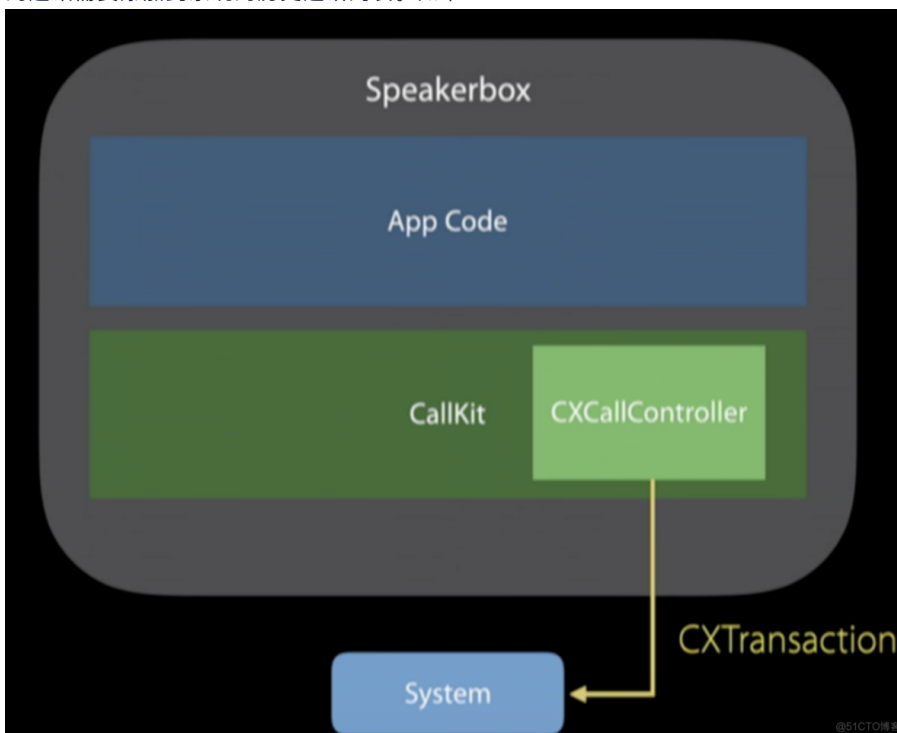
首先，App想要和系统交互，例如接收到VOIP通知后弹出通话界面，需要使用CXProvider通过CXCallUpdate来进行控制。如下图：



之后系统会将一些用户操作通过CSAction传递会APP，如下：



APP中进行的操作如果需要通知系统，需要使用CXCallController通过CXTransaction传递。例如App内的通讯需要添加到系统的历史通话列表。如下：



1.先来看CXProvider类

CXProvider类用来对系统通话界面进行一些配置操作，并处理回调逻辑，解析如下：

```
1. //初始化方法 使用CXProviderConfiguration来进行配置 后面会介绍
2. - (instancetype)initWithConfiguration:(CXProviderConfiguration *)configuration NS_D
3. - (instancetype)init NS_UNAVAILABLE;
4. //设置代理与代理函数所工作的线程
5. - (void)setDelegate:(nullable id<CXProviderDelegate>)delegate queue:(nullable dispa
6. //向系统发起一个新的通话请求
7. /*
8. UUID为此通话请求的标识 可以使用它来关闭通话
```

```

9.     update设置界面的更新参数
10.    */
11.    - (void)reportNewIncomingCallWithUUID:(NSUUID *)UUID update:(CXCallUpdate *)update
12.    //结束某个通话 使用上面的UUID作为标识
13.    /*
14.    //通话结束的原因设置
15.    typedef NS_ENUM(NSUInteger, CXCallEndedReason) {
16.        CXCallEndedReasonFailed = 1, // 通话服务失败
17.        CXCallEndedReasonRemoteEnded = 2, // 对方挂断
18.        CXCallEndedReasonUnanswered = 3, // 超时 对方为接听
19.        CXCallEndedReasonAnsweredElsewhere = 4, // 通话在其他设备接听
20.        CXCallEndedReasonDeclinedElsewhere = 5, // 通话在其他设备拒绝
21.    } API_AVAILABLE(ios(10.0));
22.    */
23.    - (void)reportCallWithUUID:(NSUUID *)UUID endedAtDate:(nullable NSDate *)dateEnded
24.    //更新通话对方的信息
25.    - (void)reportCallWithUUID:(NSUUID *)UUID updated:(CXCallUpdate *)update;
26.    //调用这个函数来进行通话呼出开始
27.    - (void)reportOutgoingCallWithUUID:(NSUUID *)UUID startedConnectingAtDate:(nullable
28.    //调用这个函数来进行通话呼出连接完成
29.    - (void)reportOutgoingCallWithUUID:(NSUUID *)UUID connectedAtDate:(nullable NSDate
30.    //配置对象
31.    @property (nonatomic, readwrite, copy) CXProviderConfiguration *configuration;
32.    //调用此函数来将通话失效
33.    - (void)invalidate;
34.    //所有未完成的事物
35.    @property (nonatomic, readonly, copy) NSArray<CXTransaction *> *pendingTransactions
36.    - (NSArray<__kindof CXCallAction *> *)pendingCallActionsOfClass:(Class)callActionCl

```

2.在看CXProviderConfiguration类

这个类用来进行Provider的配置，例如设置通讯服务名称，铃声，图标，是否支持组等。解析如下：

```

1.    //设置服务名称
2.    @property (nonatomic, readonly, copy) NSString *localizedName;
3.    //设置铃声 资源必须在 app的 bundle里
4.    @property (nonatomic, strong, nullable) NSString *ringtoneSound;
5.    //设置应用图标
6.    @property (nonatomic, copy, nullable) NSData *iconTemplateImageData;
7.    //设置最大支持的组数 默认为2
8.    @property (nonatomic) NSUInteger maximumCallGroups;
9.    //设置最大的每组人数 默认为5
10.   @property (nonatomic) NSUInteger maximumCallsPerCallGroup;
11.   //设置是否将通话记录保存进最近通话列表
12.   @property (nonatomic) BOOL includesCallsInRecents;
13.   //设置是否支持视频通话
14.   @property (nonatomic) BOOL supportsVideo;
15.   //设置支持的操作类型
16.   @property (nonatomic, copy) NSSet<NSNumber *> *supportedHandleTypes;

```

当App接收到来电VOIP通知时，可以使用CXCallUpdate来更新状态唤出通话界面。

3.CXCallUpdate类

```

1.    //远程操作对象 如果是接收方 则此为呼叫方 如果是呼叫方 则此为接收方
2.    @property (nonatomic, copy, nullable) CXHandle *remoteHandle;
3.    //名称
4.    @property (nonatomic, copy, nullable) NSString *localizedCallerName;
5.    //是否支持暂时挂起

```

```
6. @property (nonatomic) BOOL supportsHolding;
7. //是否支持组
8. @property (nonatomic) BOOL supportsGrouping;
9. //是否支持非组通话
10. @property (nonatomic) BOOL supportsUngrouping;
11. //是否支持DTMF
12. @property (nonatomic) BOOL supportsDTMF;
13. //是否包含视频
14. @property (nonatomic) BOOL hasVideo;
```

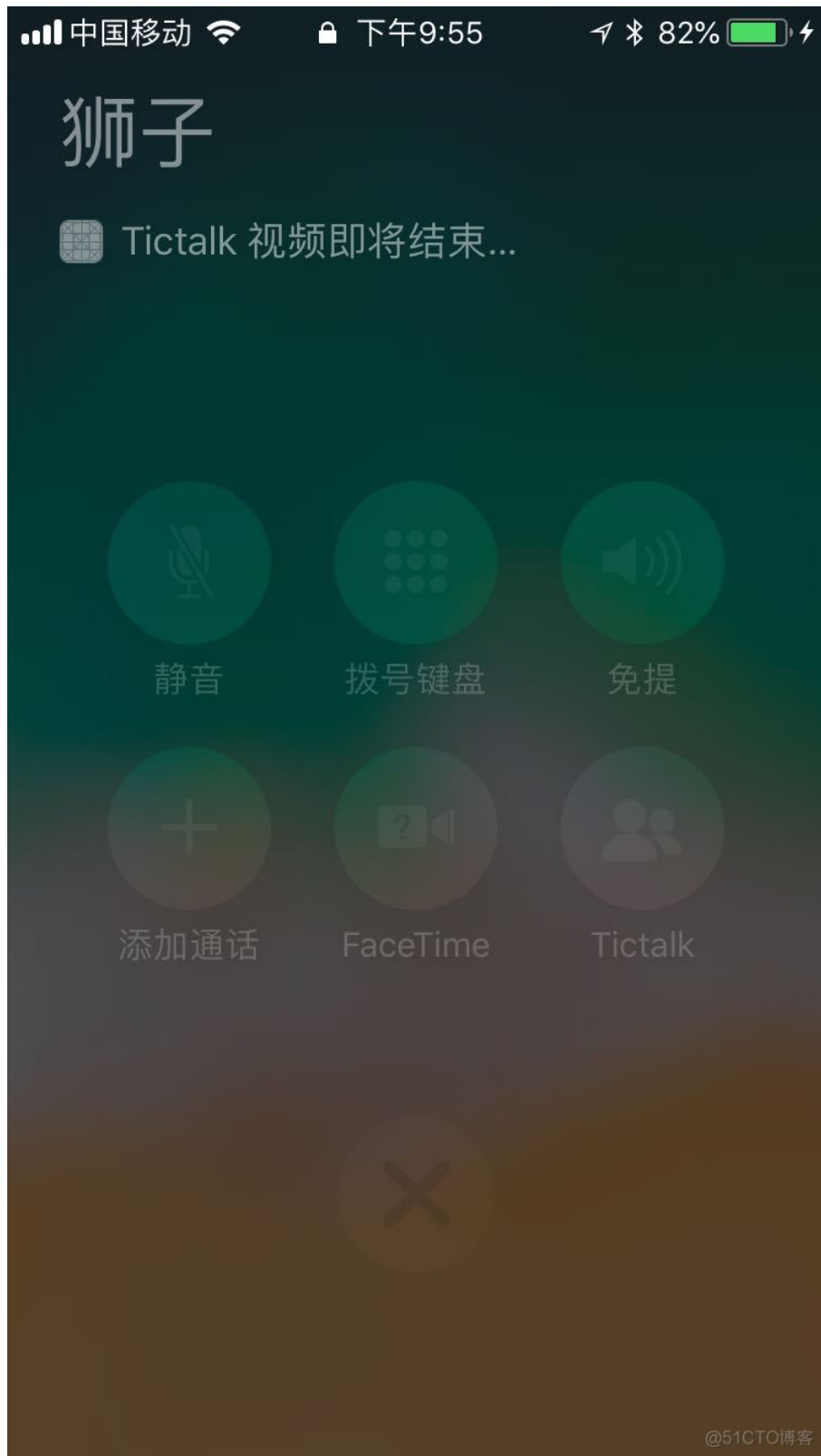
CXHandle中来定义操作的类型，解析如下：

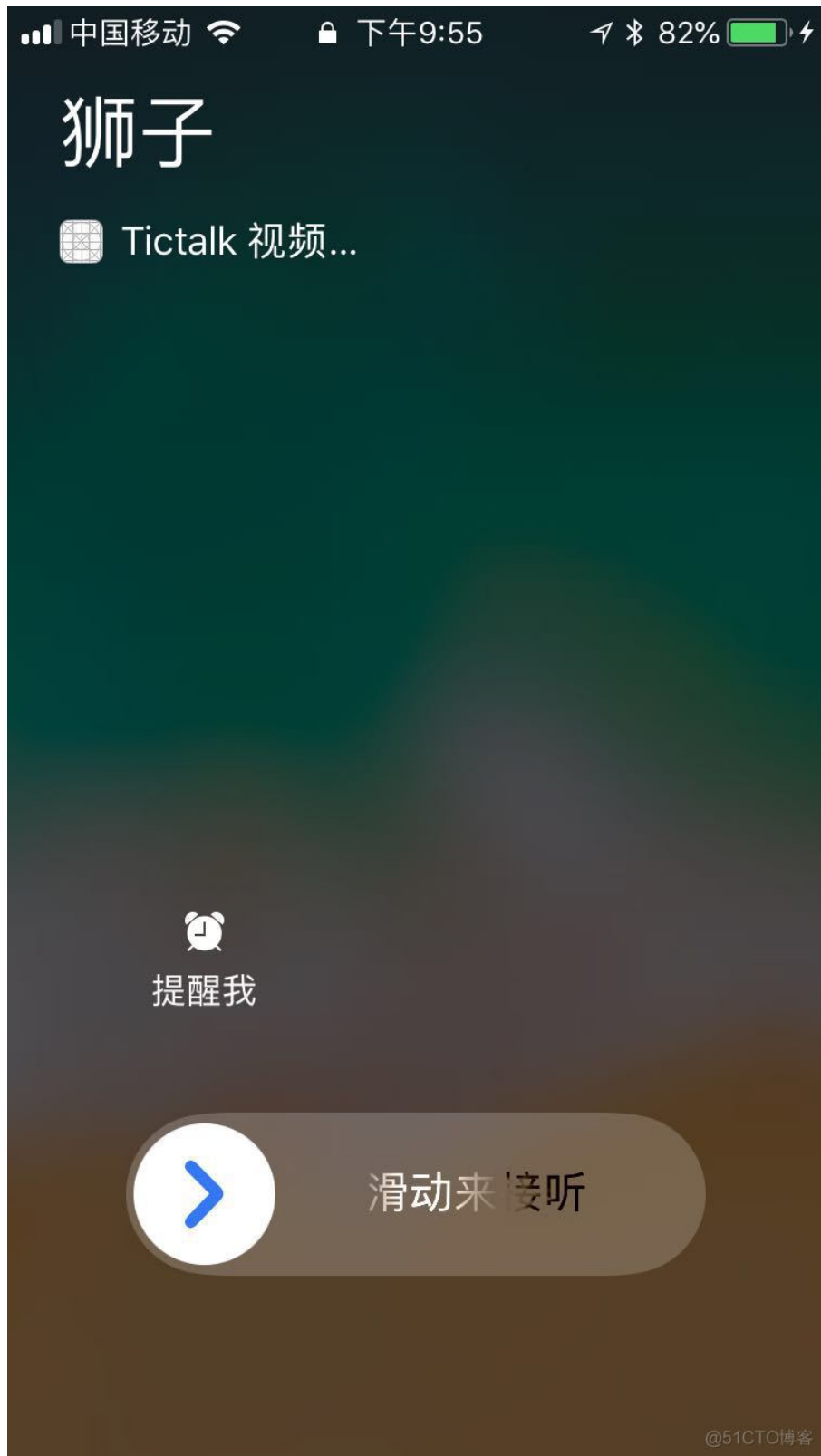
```
1. //类型
2. /*
3. typedef NS_ENUM(NSInteger, CXHandleType) {
4.     CXHandleTypeGeneric = 1, //通用
5.     CXHandleTypePhoneNumber = 2, //电话
6.     CXHandleTypeEmailAddress = 3, //邮箱地址
7. } API_AVAILABLE(ios(10.0));
8. */
9. @property (nonatomic, readonly) CXHandleType type;
10. //值
11. @property (nonatomic, readonly, copy) NSString *value;
12.
13. - (instancetype)initWithType:(CXHandleType)type value:(NSString *)value NS_DESIGNATED_INITIALIZER;
```

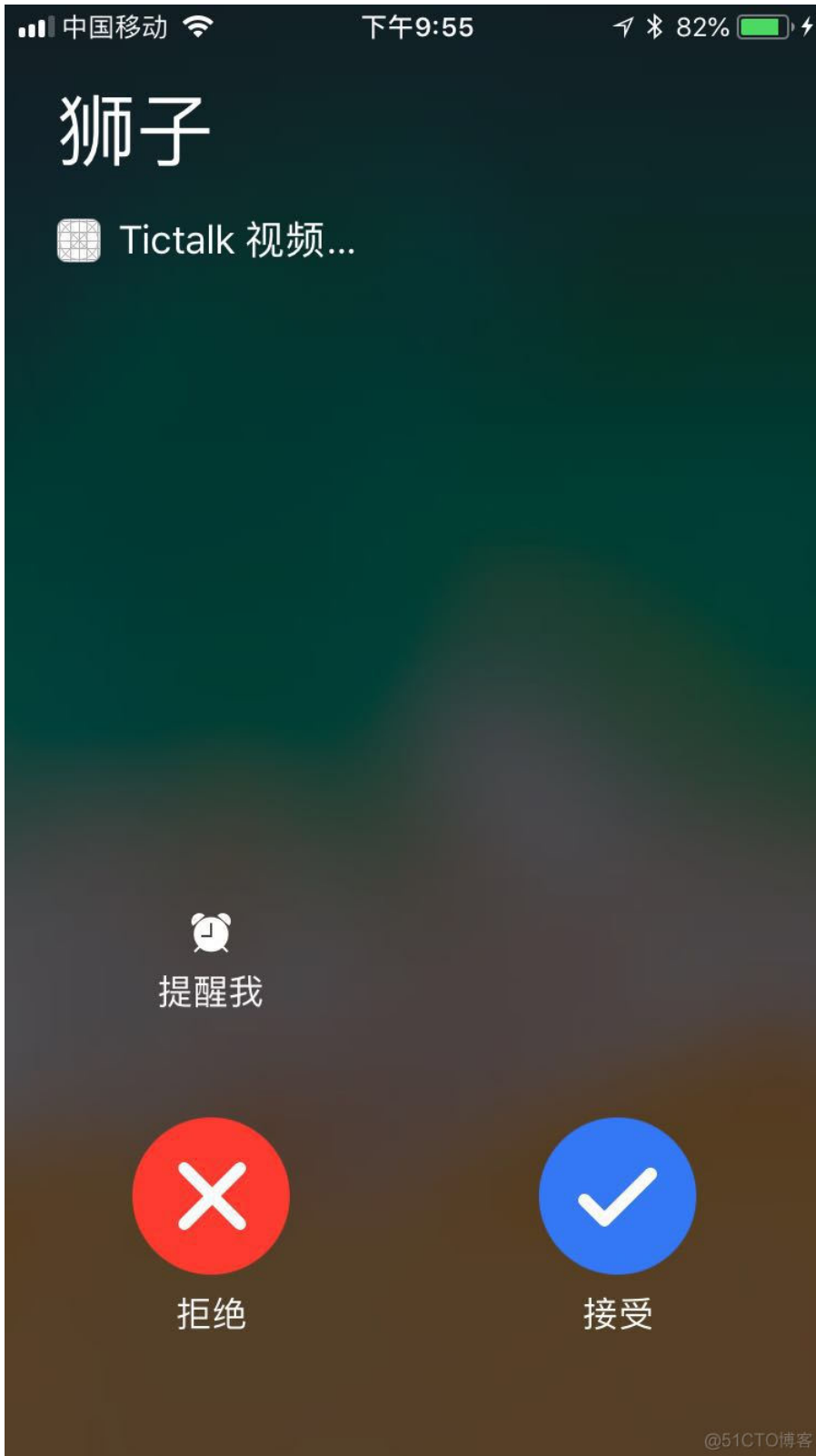
下面给出了简单的当被叫收到VOIP后调起通话界面的代码：

```
1. CXCallUpdate * callUpdate = [[CXCallUpdate alloc] init];
2. callUpdate.supportsGrouping = YES;
3. callUpdate.supportsDTMF = YES;
4. callUpdate.hasVideo = YES;
5. callUpdate.supportsHolding = YES;
6. [callUpdate setLocalizedCallerName:nickName];
7. CXHandle * handle = [[CXHandle alloc] initWithType:CXHandleTypePhoneNumber value:fromNumber];
8. callUpdate.remoteHandle = handle;
9. [[self sharedInstance].callProvider reportNewIncomingCallWithUUID:[self sharedInstance].callUUID LOG(@"吊起界面");
10. LOG(@"吊起界面");
11. }];
```

锁屏和应用程序在后台的效果分别如下所示：







4.CXProviderDelegate相关函数解析

CXProviderDelegate中的相关函数用来处理系统通话界面的某些操作回调给应用程序。

1. //当接收到呼叫重置时 调用的函数，这个函数必须被实现，其不需做任何逻辑，只用来重置状态
2. - (void)providerDidReset:(CXProvider *)provider;
3. //呼叫开始时回调
4. - (void)providerDidBegin:(CXProvider *)provider;
5. //音频会话激活状态的回调
6. - (void)provider:(CXProvider *)provider didActivateAudioSession:(AVAudioSession *)a

```

7. //音频会话停用的回调
8. - (void)provider:(CXProvider *)provider didDeactivateAudioSession:(AVAudioSession *)session;
9. //行为超时的回调
10. - (void)provider:(CXProvider *)provider timedOutPerformingAction:(CXAction *)action;
11. //有事务被提交时调用
12. //如果返回YES 则表示事务被捕获处理 后面的回调都不会调用 如果返回NO 则表示事务不被捕获, 会回调后面
13. - (BOOL)provider:(CXProvider *)provider executeTransaction:(CXTransaction *)transaction;
14. //点击开始按钮的回调
15. - (void)provider:(CXProvider *)provider performStartCallAction:(CXStartCallAction *)action;
16. //点击接听按钮的回调
17. - (void)provider:(CXProvider *)provider performAnswerCallAction:(CXAnswerCallAction *)action;
18. //点击结束按钮的回调
19. - (void)provider:(CXProvider *)provider performEndCallAction:(CXEndCallAction *)action;
20. //点击保持通话按钮的回调
21. - (void)provider:(CXProvider *)provider performSetHeldCallAction:(CXSetHeldCallAction *)action;
22. //点击静音按钮的回调
23. - (void)provider:(CXProvider *)provider performSetMutedCallAction:(CXSetMutedCallAction *)action;
24. //点击组按钮的回调
25. - (void)provider:(CXProvider *)provider performSetGroupCallAction:(CXSetGroupCallAction *)action;
26. //DTMF功能回调
27. - (void)provider:(CXProvider *)provider performPlayDTMFCallAction:(CXPlayDTMFCallAction *)action;

```

需要注意, 上面的最后几个回调中CXStartCallAction都会提供一个fullfill的函数, 当处理完成回调逻辑后, 开发者需要手动调用此函数来通知系统。同样, 其中还有一个fail和timeout函数, 调用它要通知系统此行为执行失败和超时。

5.CXCallController解析

当用户在应用程序内部进行的通讯操作时, 可以使用这个类来通知系统。

```

1. //初始化方法
2. - (instancetype)init;
3. - (instancetype)initWithQueue:(dispatch_queue_t)queue;
4. //通讯监听
5. @property (nonatomic, readonly, strong) CXCallObserver *callObserver;
6. //发起一个事务请求 CXProvider之后会接收到请求 进行逻辑
7. - (void)requestTransaction:(CXTransaction *)transaction completion:(void (^)(NSError *error))completion;
8. //通过行为发起事务
9. - (void)requestTransactionWithActions:(NSArray<CXAction *> *)actions completion:(void (^)(NSError *error))completion;
10. - (void)requestTransactionWithAction:(CXAction *)action completion:(void (^)(NSError *error))completion;

```

6.CXTransaction类

CXTransaction是封装了行为的事务。

```

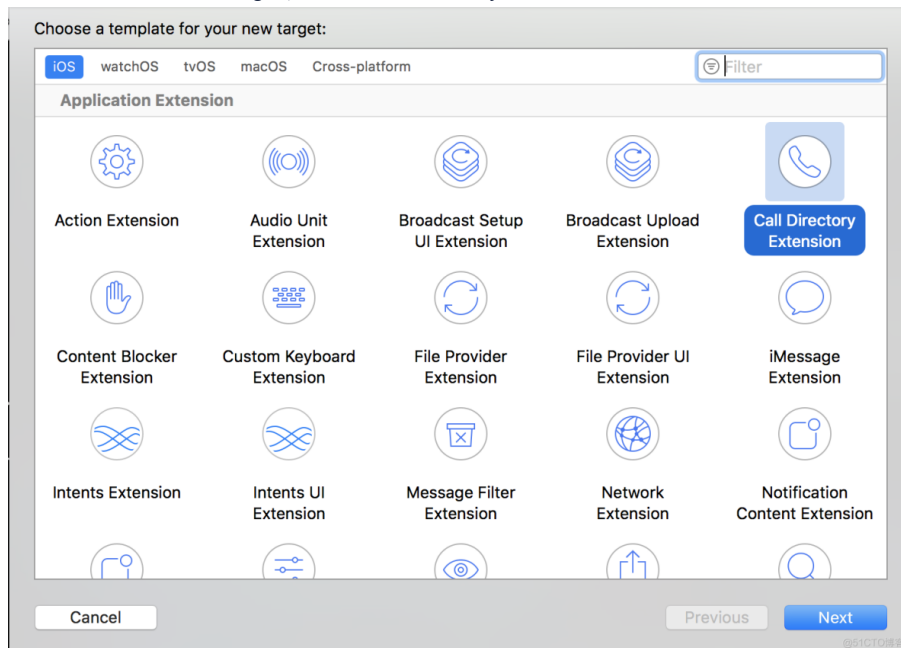
1. //唯一 ID
2. @property (nonatomic, readonly, copy) NSUUID *UUID;
3. //行为完成后的回调
4. @property (nonatomic, readonly, assign, getter=isComplete) BOOL complete;
5. //行为数组
6. @property (nonatomic, readonly, copy) NSArray<__kindof CXAction *> *actions;
7. //初始化函数
8. - (instancetype)initWithActions:(NSArray<CXAction *> *)actions;
9. - (instancetype)initWithAction:(CXAction *)action;
10. //添加行为
11. - (void)addAction:(CXAction *)action;

```

四、进行来电拦截与号码识别

上面我们介绍了使用CallKit框架来实现的通讯功能，有通讯功能就难免需要进行联系人识别与黑名单。CallKit框架中还有一部分内容可以结合Call Directory Extension来实现号码拦截与识别。

首先创建一个扩展Target，选择Call Directory Extension：



创建好Target工程后，其实需要的核心代码Xcode已经帮我们都生成。

第一步，需要在主APP中进行号码服务的验证和更新，

```
1.  _manager = [[CXCallDirectoryManager alloc] init];
2.  [_manager getEnabledStatusForExtensionWithIdentifier:@"jaki.CallKitTest.Ex" completionHandle:^(CXCallDirectoryEnabledStatus *status, NSError *error) {
3.      if (enabledStatus==CXCallDirectoryEnabledStatusEnabled) {
4.          NSLog(@"允许");
5.      }else{
6.          NSLog(@"请开启");
7.      }
8.  }];
9.  [_manager reloadExtensionWithIdentifier:@"jaki.CallKitTest.Ex" completionHandle:^(CXCallDirectoryEnabledStatus *status, NSError *error) {
10.     NSLog(@"刷新配置");
11.  }];
```

通常情况下，当用户在主APP中进行添加联系人，登录，切换账户等操作后，需要通知扩展程序进行号码库的更新，当然，一般在号码库更新时 needed 从主APP传递数据给扩展，我们可以通过Group来实现，这里不再展开。

工程运行后，会在用户的“设置->电话->来电组织与身份识别”项目中看到扩展程序：



阻止联系人...

您不会收到黑名单上联系人的来电、信息或 FaceTime 通话。

@51CTO博客

当用户打开此服务或者调用上面的reloadExtension时，会从执行扩展程序的相关方法来重新加载号码库。
需要注意，reloadExtension函数中的id参数为扩展项目的bundleID，不是主项目的。
在扩展工程的info.plist文件中，默认配置好了处理来电的操作类，如果要自定义，需要开发者手动修改：

▼ NSExtension	Dictionary (2 items)
NSExtensionPointIdentifier	String com.apple.callkit.call-directory
NSExtensionPrincipalClass	String CallDirectoryHandler

@51CTO博客

默认的CallDirectoryHandler类为来电拦截与身份识别的操作类，其集成自CXCallDirectoryProvider类，当收到加载号码库的请求时，会执行下面的函数：

```
1. - (void)beginRequestWithExtensionContext:(CXCallDirectoryExtensionContext *)context
```

```

2.     context.delegate = self;
3.     //是否支持增量更新
4.     if (context.isIncremental) {
5.         [self addOrRemoveIncrementalBlockingPhoneNumbersToContext:context];
6.
7.         [self addOrRemoveIncrementalIdentificationPhoneNumbersToContext:context];
8.     } else {
9.         [self addAllBlockingPhoneNumbersToContext:context];
10.
11.        [self addAllIdentificationPhoneNumbersToContext:context];
12.    }
13.    //完成更新操作
14.    [context completeRequestWithCompletionHandler:nil];
15. }

```

上面是Xcode默认提供的实现，十分优雅，在iOS11后，号码库的更新支持增量，所以这里进行的区分。

CXCallDirectoryExtensionContext是一个操作上下文，通过它可以像号码库中添加删除数据。解析如下

:

```

1.     //是否支持增量更新
2.     @property (nonatomic, readonly, getter=isIncremental) BOOL incremental API_AVAILABLE(
3.     //添加一个黑名单号码
4.     - (void)addBlockingEntryWithNextSequentialPhoneNumber:(CXCallDirectoryPhoneNumber)p
5.     //移除一个黑名单号码
6.     - (void)removeBlockingEntryWithPhoneNumber:(CXCallDirectoryPhoneNumber)phoneNumber
7.     //移除所有的黑名单号码
8.     - (void)removeAllBlockingEntries API_AVAILABLE(ios(11.0));
9.     //添加一个身份识别
10.    - (void)addIdentificationEntryWithNextSequentialPhoneNumber:(CXCallDirectoryPhoneNu
11.    //移除一个身份识别
12.    - (void)removeIdentificationEntryWithPhoneNumber:(CXCallDirectoryPhoneNumber)phoneN
13.    //移除所有身份识别
14.    - (void)removeAllIdentificationEntries API_AVAILABLE(ios(11.0));
15.    //完成操作后 需要手动调用此函数
16.    - (void)completeRequestWithCompletionHandler:(nullable void (^)(BOOL expired))compl

```

添加了黑名单后，用户将收不到此号码的电话，同样，设置了身份识别后，当用户播出前，会显示设置的身份信息(需要注意，大陆号码需要前面带86)，如下：

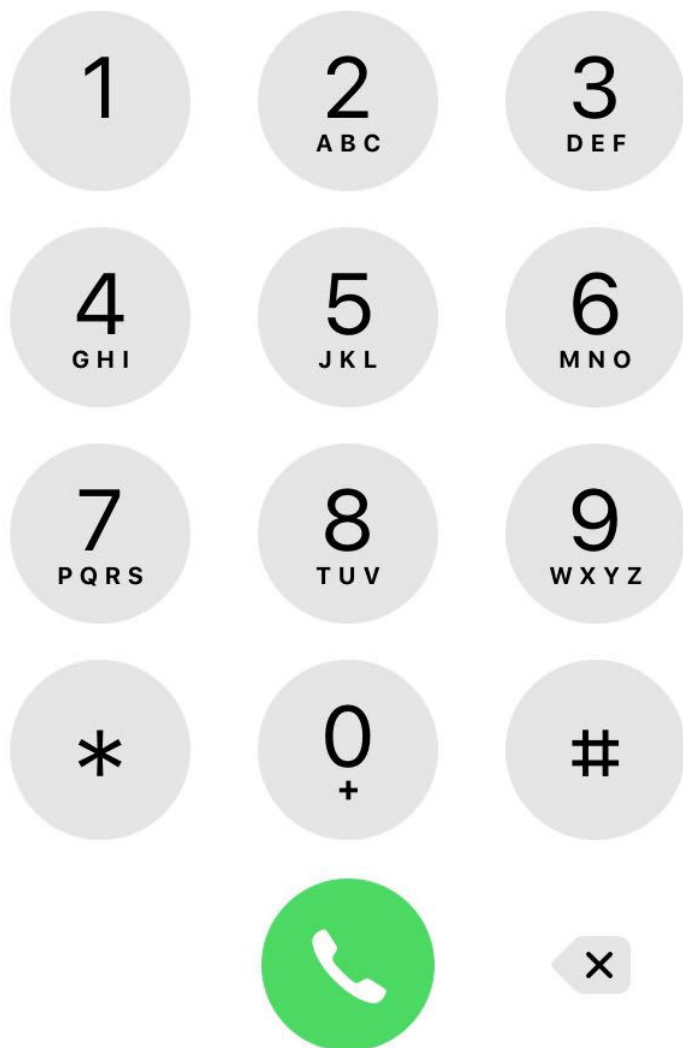
中国移动 4G

下午3:04

100% 

188 8555 5555

CallKitTest: Local business

[上一篇：从一些简单的例子看算法时间复杂度](#)[下一篇：iOS开发中Log模块的解决方案](#)



提问和评论都可以，用心的回复会被更多人看到

评论

相关文章

Swift Callkit扩展--来电标记

需求：需要实现项目中根据号码从服务器查询返回的标记写入callkit放骚扰标记系统库中，来电时显示该标记最终的效果如图...

App Groups 防骚扰标记 Swift ios

iOS使用VOIP与CallKit实现体验优质的网络通讯功能

4G时代为移动网络性能带来的极大的提升，通过网络进行音频视频通话更加方便。在iOS系统中，可以配合使用VOIP与CallKit...

CallKit VoIP iOS网络电话 iOS识别联系人 iOS电话黑名单

【音视频】Android CallKit 开发指南

简介CallKit 是融云音视频通话功能的 UI 界面 SDK。包含了单人、多人音视频通话的界面的各种场景和功能。您可以快速的集...

SDK 音视频 即时通讯

Swift : 在VoIP应用中接入iOS10中的CallKit框架--即接听电话使用系统原生界面

由于目前开发的应用是一款VoIP应用，需要接入iOS10新框架CallKit，由于这个框架在官网也只有各种方法，没有具体的说明...

swift voip 界面 iOS10 CallKit框架

iOS: UIWindow in iOS

原文链接：<http://www.fantageek.com/1317/uiwindow-in-ios/>一篇中文翻译：点击打开链接In this article, I 'll share what I kno...

iOS UIWindow KeyWindow ide ios

【iOS】发布iOS应用

// 登录iTunes Connect<https://itunesconnect.apple.com/WebObjects/iTunesConnect.woa/> 点击 My App, 添加一个 New iOS ...

iOS distribution pub

【iOS】iOS main() 简介

C 语言编写的程序，其执行入口都是 main()。用 Objective-C 语言编写的程序也是这样。main.m 中的代码如下：代码中的 UI...

运行循环 初始化 ios 类名 objective-c

iOS学习——Hello iOS（二）

前面写了iOS开发环境搭建，只简单提了一下安装Xcode，这里再补充一下，点击下载Xcode的dmp文件，稍等片刻会有图...

ios

iOS 兼容iOS7.0

1、iOS7.0新增加的icon尺寸76 x 76：Size for iPad 2 and iPad mini (standard resolution)120 x 120：Size for iPhone and iPo...

iOS7 新特性

【IOS下载】Cisco IOS下载

Cisco IOS下载(新添加3745 2691 3725)资源都是网上搜集，cisco不要找我~再告诉大家一个办法，用eMule,打开后点搜索栏...

Cisco 职场 下载 IOS 休闲

ios

Undefined symbols for architecture arm64: "_OBJC_CLASS_\$_AVCaptureAudioDataOutput", referenced from: objc-cla...

Mac iOS

iOS Graphics Technologies iOS 图形技术

High-quality graphics are an important part of all iOS applications. The simplest (and most efficient) way to create an app...

天翼空间 iOS开发

iOS学习——iOS系统架构（三）

iOS的系统架构分为四个层次：核心操作系统层（Core OS layer）、核心服务层（Core Services layer）、媒体层（Medi...

ios iOS系统架构

iOS iOS8注册通知

目前分为四个推送：用户推送，本地推送，远程推送，地理位置推送。用户推送我们先开始讲这个用户推送,我们要使用之前必...

程序 用户 推送 ios8

iOS SDK：iOS调试的技巧

iOS SDK：iOS调试的技巧 为什么你的数组包含3个项目而不是5个？为什么你的游戏运行缓慢？这些都跟调试有关，调试...

ios 调试 SDK

iOS---iOS9搜索功能

前言 在iOS9之前我们只能使用Spotlight来搜索应用名称来打开指定App，而其他的内容都是提供给系统使用(信息，联系人， ...

ios---iOS9搜索功能

【iOS】iOS viewDidLoad 方法名问题

这两天在调试一个项目，跳转到一个页面的时候总是不显示标题栏（当然也没有标题栏的返回按钮），搞了好久，今天总算找...

标题栏 开发人员 方法名 返回按钮 iOS

【iOS】containsString iOS7 报错

前几天发现了这个问题，原来是因为containsString 只支持 iOS8.0 以后的系统，不支持 7...有些地方可以用其他方法替代，如...

ios 解决方案 IOS